

PragLocker: Protecting Agent Intellectual Property in Untrusted Deployments via Non-Portable Prompts

Qinfeng Li^{1*} Yuntai Bao^{1*} Jianghui Hu^{2*} Wenqi Zhang³ Jintao Chen¹ Huifeng Zhu⁴ Yier Jin⁵
Xuhong Zhang³✉

Abstract

LLM agents rely on prompts to implement task-specific capabilities based on foundation LLMs, making agent prompts valuable intellectual property. However, in untrusted deployments, adversaries can copy and reuse these prompts with other proprietary LLMs, causing economic losses. To protect these prompts, we identify four key challenges: proactivity, runtime protection, usability, and non-portability that existing approaches fail to address. We present PragLocker, a prompt protection scheme that satisfies these requirements. PragLocker constructs function-preserving obfuscated prompts by anchoring semantics with code symbols and then using target-model feedback to inject noise, yielding prompts that only work on the target LLM. Experiments across multiple agent systems, datasets, and foundation LLMs show that PragLocker substantially reduces cross-LLM portability, maintains target performance, and remains robust against adaptive attackers.

 [Code](#)

1. Introduction

In recent years, amid the rapid adoption of large language models (LLMs), intelligent LLM agents, such as Cursor (Cursor, 2025), Manus (Manus AI, 2026), and Zapier (Zapier, 2025), have emerged as autonomous executors of complex tasks. These agents, serving as a key interface between LLM capabilities and real-world applications, implement task capabilities largely through an agent *system*

prompt that specifies tasks, policies, and tool-use on top of a foundation LLM (OpenAI, 2025b; Anthropic, 2025). As a result, prompt design becomes a key determinant of agent behavior and performance, constituting a core competitive asset. In particular, many agents may invoke the same underlying LLM (e.g., proprietary ones like ChatGPT or Gemini) (OpenAI, 2022; Team et al., 2023), yet yield substantially different functionality and quality.

However, as highly valuable intellectual property (IP), prompts are vulnerable to theft after deployment, leading to substantial economic losses. Specifically, crafting a high-quality agent prompt typically requires significant expert knowledge and continual real-world iteration, making it a highly valuable asset (Sahoo et al., 2024; Yang et al., 2025a). However, in practice, agents often run on user devices (Cursor, 2025), cloud services (Spector, 2025), or multi-tenant infrastructures (Cloud, 2011), where adversaries (e.g., malicious end users or insider cloud operators) may copy and misuse the prompt (Hui et al., 2024; Wang et al., 2024a). Worse still, prompts are typically written in natural language; once leaked, they can be easily reused on any other, even stronger, proprietary LLM. *Once leaked, an attacker can reuse the prompt with other LLMs to build a similar or even stronger agent, undermining the original agent’s competitive advantage and causing substantial losses.* Therefore, effectively protecting these prompts in the deployment of agent systems has become a critical issue.

Unfortunately, as shown in Table 1, traditional solutions struggle to protect the prompt of untrusted environment-deployed agents as they fail to satisfy the diverse requirements. First, passive protection methods, e.g., **prompt watermarking** (Yang et al., 2025b; Yao et al., 2024), primarily verify ownership *after* misuse. Thus, they do not proactively prevent prompt theft, rendering these methods vulnerable: once a prompt is stolen, it may be freely exploited without detection. In contrast, proactive protections aim to prevent unauthorized usage. For example, **encryption-based protection** (Rana et al., 2023; The Kubernetes Authors, 2025)) keeps confidential information encrypted during transmission, distribution, and storage. However, at runtime, the prompt must be submitted to black-box LLM API (OpenAI,

*Equal contribution. ✉Corresponding author. ¹Zhejiang University ²Chang’an University ³Innovation and Management Center, School of Software Technology (Ningbo), Zhejiang University ⁴Washington University in St. Louis ⁵University of Science and Technology of China. Correspondence to: Xuhong Zhang <zhangxuhong@zju.edu.cn>.

Table 1. Comparison with existing solutions. ✓/✗ illustrates whether the method can achieve the corresponding property.

Solutions (exemplar)	Proactivity	Runtime security	Usability	Non-Portability
Prompt Watermarking (Yang et al., 2025b)	✗	✓	✓	✗
Encryption-based Protection (The Kubernetes Authors, 2025))	✓	✗	✓	✗
Prompt Obfuscation (Pape et al., 2025)	✓	✓	✗	✗
PragLocker (ours)	✓	✓	✓	✓

2025a; Google, 2025) in plaintext; therefore, encryption can only protect the prompt at rest, not during inference.

Alternatively, a potential method is **prompt obfuscation** (Pape et al., 2025), which replaces the original system prompt with an obfuscated yet function-preserving variant. However, existing prompt obfuscation techniques are not directly applicable to agent prompt protection: they either remain portable across LLMs or require white-box access to the underlying model. Specifically, EmojiPrompt (Lin et al., 2025) encodes prompts into emojis to degrade human readability, but other LLMs can still decode and follow such surface-level encodings, so it does not prevent cross-model reuse. A closer line of work constructs prompts that appear noise-like and less portable while preserving utility on a target model; e.g., Pape et al. (2025) obfuscates prompts via reversible semantic transformations in the model’s representation space. However, many real-world agents (e.g., Cursor and Copilot) rely on proprietary black-box LLMs, where developers only have API-level access, making such white-box methods inapplicable. Fundamentally, *constructing a prompt that is usable on the target LLM yet non-portable is challenging*, especially in black-box settings, where one can only rely on input-output feedback. The problem is further exacerbated by intra-family transfer: intra-family LLMs often behave similarly (McGovern et al., 2025), further weakening the behavioral separation needed for non-portability.

Considering the limitations of existing defense strategies, we identify four key challenges (C) in protecting prompts for agents deployed in untrusted environments. **C1** (Proactivity): ensuring the prompt cannot be misused even if physically obtained by an attacker. **C2** (Runtime protection): ensuring protection not only during deployment but also during inference. **C3** (Usability): the protected prompt must preserve the agent’s performance on the target LLM. **C4** (Non-Portability): simultaneously, the protected prompt must be human-unintelligible and ineffective on other LLMs.

In this paper, we propose **PragLocker** (Agent Prompt Locker), a black-box prompt obfuscation scheme. PragLocker obfuscates the system prompt before deployment, so the original prompt is never released in plaintext (addressing **C1**). This obfuscated prompt remains behavior-preserving on the target LLM and can be used directly without deobfuscation (addressing **C2**).

PragLocker is grounded in a key question: *Does there exist such an obfuscated prompt that preserve same behavior on a target LLM while failing on other LLMs?* We answer yes with a theoretical motivation: there exists a perturbed prompt that makes the target LLM produce the same next token while causing a different next token on other LLMs.

Despite our theoretical insights, practical construction remains challenging. To obtain such prompts under the black-box constraint, PragLocker proceeds in two phases. First, it performs an *initialization transformation* that converts the prompt into a code-symbol form, yielding an initial non-natural-language, utility-preserving concealment. However, this transformed prompt does not yet satisfy our goals: it can still be understood by other LLMs (many models can interpret code and symbols). To address this issue, PragLocker introduces a second phase, *noise-injected prompt optimization*. Specifically, analogous to black-box discrete optimization, we progressively inject character-level noise into the prompt as the optimization to reproduce the target LLM’s original outputs. As a result, the noise is guided solely by target-LLM feedback; the resulting obfuscated prompt becomes model-specific: it preserves performance on the target LLM (addressing **C3**) while appearing noise-like and ineffective on other LLMs (addressing **C4**).

We evaluate PragLocker across diverse agent systems, foundation LLMs, and tasks, demonstrating strong protection without sacrificing task performance. Our obfuscated prompts exhibit near-zero portability across LLMs, even between FP16 and 4-bit variants of the same LLM. Moreover, *even target LLM itself* cannot reliably interpret the obfuscated prompt, which instead behaves like a *model-conditioned trigger* without retaining recoverable text-level information. The contributions of this work are as follows:

- To our knowledge, we are the first to study prompt protection for agents under untrusted deployment, where adversaries may copy and obtain the deployed prompt. We identify prompts as the primary embodiment of agent IP and distill four requirements for protection.
- We propose PragLocker, a black-box prompt protection scheme that replaces the system prompt with an obfuscated prompt that works on the target LLM but fails on others. We further provide a theoretical motivation by proving an existence theorem for such model-specific, function-preserving obfuscated prompts.

- To construct such prompts under pure black-box access, PragLocker uses a two-phase pipeline: an *initialization transformation* followed by *noise injection*, optimizing obfuscated prompts without model internals.
- Experiments across agent systems, datasets, and LLMs show that PragLocker preserves target performance, sharply reduces cross-model portability, and withstands adaptive attacks. We also report ablations and case studies to validate key design choices.

2. Preliminaries

2.1. LLM Agent

LLM agents (Wang et al., 2024b) are task-oriented systems built on top of foundation LLMs. Concretely, an agent is mainly defined by (i) a *system prompt* that specifies tasks, policies, and tool-use, and (ii) an underlying LLM that executes prompt-conditioned reasoning and generation. In practice, many commercial agents (Cursor, 2025; Friedman, 2021) are built on the same proprietary black-box foundation models, *making the system prompt the dominant portion of the agent developer’s IP*. In addition, agents are frequently deployed in heterogeneous and potentially untrusted environments (Cloud, 2011; Spector, 2025), including third-party hosting platforms, multi-tenant clouds, and end-user devices, which increases the exposure surface of the deployed prompt and elevates prompt confidentiality and integrity as central security concerns.

2.2. Related Work

Prompt Watermarking. Prompt watermarking embeds a signature into a prompt to provide ownership evidence, enabling verification after theft. For example, PromptCARE (Yao et al., 2024) applies small, utility-preserving perturbations and detects infringement via a predefined verification protocol (e.g., trigger queries) that yields statistically distinguishable response patterns. More recent work (e.g., PromptCOS (Yang et al., 2025b)) moves toward content-only auditing by jointly optimizing the system prompt with verification queries and target “signal” outputs, then testing suspected prompts via output-similarity based verification.

Encryption-based Protection. Conventional encryption (e.g., storing the system prompt as an encrypted file/secret (The Kubernetes Authors, 2025)) protects confidentiality at rest and in transit, but inference requires decryption into plaintext and transmission to the LLM, allowing the execution host to extract the prompt post-decryption (Karvandi et al., 2024). TEEs offer attested isolation (e.g., SGX/TDX (Costan & Devadas, 2016), GPU confidential computing (Nertney, 2023)) to reduce runtime exposure, yet are often incompatible with LLM agents: many rely on

proprietary black-box API LLMs (OpenAI, 2025a; Google, 2025), so the model cannot run inside the same trusted environment and the decrypted prompt must still be sent to external services, undermining confidentiality.

Confidentiality Infrastructure. Confidential-inference infrastructure protects prompt privacy by providing an attested TEE/CVM endpoint so prompts are decrypted and processed only inside a protected runtime, sometimes with split/partitioned execution to maintain serving efficiency (Gim et al., 2024; Yuan et al., 2025; Su & Zhang). However, it requires third-party platform cooperation (attestation/key release, correct routing) and thus is outside our threat model: a malicious platform can still access the prompt in plaintext.

Prompt Obfuscation. Prompt obfuscation replaces the original system prompt with an obfuscated yet function-preserving variant. For example, EmojiPrompt (Lin et al., 2025) encodes sensitive text into emoji-based representations. However, such surface-level obfuscation does not prevent *reuse on other LLMs*, offering no defense against cross-model misuse. Pape et al. (2025) optimizes prompts in the model’s representation space to retain utility while concealing the underlying instructions. These methods, however, require model-internal access, which is unavailable in black-box agent deployments.

Prompt Optimization. *Soft prompt tuning* is a well-founded approach to parameter-efficient fine-tuning, which trains task-specific embeddings as prompt prefix to task queries using gradient-based optimization (Lester et al., 2021). Historically, discrete prompt optimization is a difficult problem (Shin et al., 2020; Singh et al., 2023) and gradient-free optimization is even harder (Deng et al., 2022; Zhang et al., 2024). We position our technical objective as gradient-free discrete optimization that optimizes for task performance, obfuscation, and non-portability constraints.

2.3. Threat Model

We consider two parties: the defender is the party that owns the deployed agent, and the attacker aims to steal the prompt.

Defender. The defender deploys an LLM agent as a service (e.g., on third-party hosting or end-user devices) and accesses the target LLM only via a black-box API. The target LLM is trusted not to exfiltrate prompts, while the deployment environment is untrusted. The defender’s goal is to proactively provide runtime protection for the agent system prompt while preserving on-target usability, such that even if the deployed prompt is obtained, it cannot be reused to reproduce comparable behavior on non-target LLMs.

Adversary. The adversary can obtain the deployed prompt. An attack succeeds if the stolen (or recovered) prompt enables similar agent functionality on a different model. Beyond naïve copying, the adversary may use limited compute

and data to mount adaptive recovery attacks (e.g., deobfuscation optimization) to reconstruct a usable prompt.

3. Our Design: PragLocker

3.1. Problem Formulation

Let the prompt space be $\mathcal{P} \subset \{\mathcal{V}, \mathcal{V}^2, \dots\}$ where $\mathcal{V}$ is model vocabulary, the protected agent prompt be $\mathbf{x} \in \mathcal{P}$ and θ be model parameters. **Our objective** is to identify another prompt $\tilde{\mathbf{x}} \in \mathcal{P}$ that satisfy the following criteria: (1) *Obfuscation (C2)*: $\tilde{\mathbf{x}}$ is distant from $\mathbf{x}$ in prompt space; (2) *Usability (C3)*: $\tilde{\mathbf{x}}$ is functionally equivalent to $\mathbf{x}$; (3) *Non-portability (C4)*: the utility objective is satisfied only on the target LLM, not other LLMs.

3.2. Theoretical Motivation

In this subsection, we explain the theoretical motivation for the feasibility of PragLocker, i.e. to identify an alternative prompt to the original prompt that simultaneously satisfy the criteria of obfuscation, usability and non-portability. We defer detailed discussions to Appendix Section A.

Roadmap. We define *functional equivalence* for single-token generation, prove the existence of *local stability regions* that allow for obfuscation, and discuss *non-portability* across models.

Local Obfuscation and Utility. Let $\mathbf{h} = \text{Embed}(\mathbf{x}) \in \mathbb{R}^{n \times d}$ be the embeddings for prompt $\mathbf{x}$. We seek an obfuscated prompt $\tilde{\mathbf{x}}$ with embeddings $\tilde{\mathbf{h}} = \mathbf{h} + \delta$ such that output behavior is preserved under greedy decoding.

Definition 3.1 (Functional equivalence). Embeddings $\tilde{\mathbf{h}}$ and $\mathbf{h}$ are equivalent w.r.t. query $\mathbf{q}_i$ if $\arg \max_y f(y|\tilde{\mathbf{h}}, \mathbf{q}_i) = \arg \max_y f(y|\mathbf{h}, \mathbf{q}_i)$.

Trivially, equivalence holds if the *correct-class margin* $m(\tilde{\mathbf{h}}, \mathbf{q}_i, y_i) := f(\tilde{\mathbf{h}}, \mathbf{q}_i)_{y_i} - \max_{k \neq y_i} f(\tilde{\mathbf{h}}, \mathbf{q}_i)_k$ remains positive. Since $f(\cdot; \theta)$ is a composition of continuous functions (layers, activations), $m(\cdot)$ is continuous. Thus, for any $\mathbf{h}$ where $m > 0$, there exists an ϵ -ball $B_\epsilon(\mathbf{h})$ within a *stability region* $S_{\mathbf{x}} = \{\mathbf{h}' \mid \forall \mathbf{q}_i \in \mathcal{Q}, m(\mathbf{h}', \mathbf{q}_i, y_i) > 0\}$ where utility is perfectly preserved.

Theorem 3.2 (Existence of obfuscated prompts). *For a prompt $\mathbf{x}$, there exists $\tilde{\mathbf{x}} \neq \mathbf{x}$ such that $\tilde{\mathbf{h}} \in S_{\mathbf{x}}$ (Utility) and $d(\tilde{\mathbf{x}}, \mathbf{x}) \geq d_0$ (Obfuscation).*

Proof sketch. Transformer attention often exhibits low sensitivity to specific tokens (attention dilution). By perturbing k such tokens, the cumulative shift $\|\Delta \mathbf{h}\| \leq \sum_{j \in \mathcal{K}} \|\delta_j\|$ can be kept within ϵ to maintain utility, while the distance $d(\tilde{\mathbf{x}}, \mathbf{x})$ grows with k to satisfy the obfuscation bound d_0 .

Non-portability. Non-portability arises from *manifold mismatch*: the stability region $S_{\mathbf{x}}(\theta)$ is highly dependent on

the specific geometry of the loss landscape defined by θ . A perturbation δ optimized to stay within the decision boundaries of model θ is unlikely to reside within the distinct stability region $S_{\mathbf{x}}(\theta')$ of a different model, as $P(\tilde{\mathbf{h}} \in S_{\mathbf{x}}(\theta')) \ll P(\tilde{\mathbf{h}} \in S_{\mathbf{x}}(\theta))$ in high-dimensional space.

Remark (Between our theoretical motivation and method design). Although our methodology is not entirely aligned with our discussions above, we point out that these analyses serve as a theoretical lens that motivates our method design, showcasing the possibility of identifying prompts that satisfy the obfuscation, utility, and non-portability criteria.

3.3. PragLocker Methodology

Overview. As is shown in Figure 1, PragLocker consists of two components: prompt initialization and obfuscation optimization. Prompt initialization primarily contributes to our utility objective, while obfuscation optimization is used to achieve obfuscation and non-portability objectives.

Prompt Initialization. Our prompt initialization strategy consists of two key design considerations. First, we use the original prompt $\mathbf{x}$ as the warm-start, which is essential for ensuring that the obfuscated prompt achieves the same level of performance as the original task prompt. Second, we prompt the target LLM to encode $\mathbf{x}$ in a code-symbol form, such that the encoded prompt ($\tilde{\mathbf{x}}_0$) is no longer written in natural language while having the same semantics as $\mathbf{x}$. This approach could be understood as a preliminary obfuscation step conditioned by the target LLM that also creates redundancy, thereby setting the stage for subsequent obfuscation optimization.

Non-Portable Obfuscation Optimization. Our obfuscation strategy is built upon *random search (RS)*, a black-box, gradient-free, discrete optimization method (Rastrigin, 1963). This approach is reminiscent of *evolution strategies* (Schwefel, 1977; Salimans et al., 2017), and has recently been used for gradient-free optimization of jailbreaking suffixes (Andriushchenko et al., 2025).

RS allows us to optimize textual prompts when there is only API-level access to prompt inputs, responses, and log-probabilities of the target LLM. The **intuition** for RS to work well in practice is that, although it is convenient for humans to describe task-solving requirements and instructions in natural language, it might not be the most effective and efficient approach (Chang et al., 2024; Li et al., 2025). Therefore, the inherent redundancy of the initialized prompt ($\tilde{\mathbf{x}}_0$) allows RS to improve model performance while leaving room for obfuscation.

We provide an algorithmic description of obfuscation optimization in Algorithm 1. Obfuscation optimization might be understood as RS over the entire task prompt $\mathbf{x}$. At each

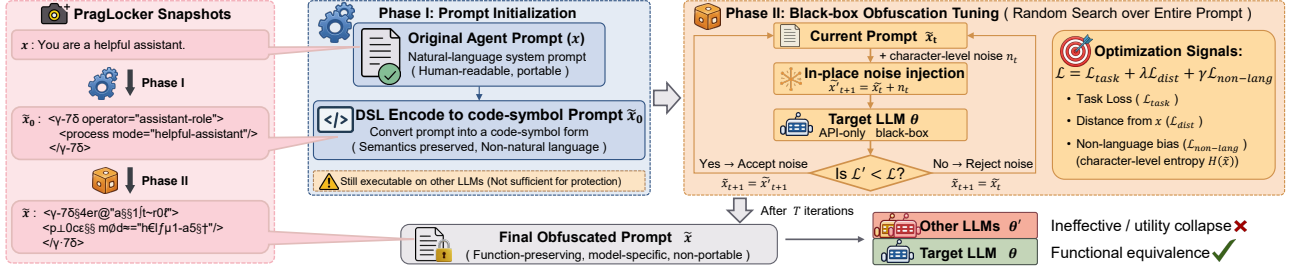


Figure 1. A pipeline of PragLocker. PragLocker transforms a plaintext prompt into a model-specific obfuscated form through a two-phase process: (i) a code-symbol initialization that preserves task semantics, and (ii) noise-injected black-box optimization driven by target LLM feedback. The final obfuscated prompt remains usable at runtime on the target LLM but resists reuse or recovery on other models.

step of obfuscation optimization, we inject textual noise from a predesignated noise set, usually the commonly used set of printable characters. Since each instance of noise might have a mixed effect on either task performance or textual readability, we constrain the accepted set of noise with a custom objective function to ensure that the final obfuscated prompt is a piece of non-natural text that retains task performance.

Optimization Objective. The objective function for obfuscation optimization consists of three terms:

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{task} + \lambda \mathcal{L}_{dist} + \gamma \mathcal{L}_{non-lang}, \text{ where} \\ \mathcal{L}_{task} &= -\log p(y|q, \tilde{x}), \\ \mathcal{L}_{dist} &= -\log \sigma(\text{Dist}(\tilde{x}, x)), \\ \mathcal{L}_{non-lang} &= -H(\tilde{x}), \end{aligned} \quad (1)$$

where $\lambda, \gamma \in \mathbb{R}$ are constant coefficients and $\sigma(\cdot)$ is the sigmoid function; $\mathcal{L}_{task}$ implements the utility constraint, $\mathcal{L}_{dist}$ implements the obfuscation constraint while $\mathcal{L}_{non-lang}$ implements both obfuscation and non-portability constraints.

Particularly, $\mathcal{L}_{dist}$ ensures that the final obfuscated prompt differs from the original prompt by maximizing their edit distance where $\text{Dist}(\cdot)$ is *Levenshtein distance* (Levenshtein, 1966); $\mathcal{L}_{non-lang}$ pushes the obfuscated prompt away from the natural language distribution by minimizing Shannon entropy: $H(\tilde{x}) = -\sum_{c \in \mathcal{A}} v_c \log v_c$ where $\mathcal{A}$ is alphabet and v_c is the frequency of character c appearing in $\tilde{x}$. $\mathcal{L}_{non-lang}$ is motivated by the fact that natural language prompts are inherently portable. By pushing the obfuscated prompt away from the natural language distribution, we make it dedicated for model-specific intricacies of the loss landscape, thereby implicitly minimizing its inter-model portability.

4. Experiments

4.1. Experimental Settings

Agents. We evaluate PragLocker across diverse agent domains by considering three representative agents with different tasks and designs: LessonL (Liu et al., 2025) (multi-agent programming with a shared lesson bank), ReadA-

Algorithm 1 PragLocker algorithm.

Input: Base LLM $p(\cdot)$, agent prompt x , task training set $\mathcal{D}$, training steps T , noise set $\mathcal{N}$, loss function $\mathcal{L}(\cdot)$
Output: Obfuscated agent prompt $\tilde{x}$

```

 $\tilde{x}_0 \leftarrow \text{Init}(x)$  {Initialization}
 $t \leftarrow 0$ 
while  $t < T$  do
     $(q_t, y_t) \sim \mathcal{D}$ 
     $n_t \sim \mathcal{N}$  {Sample character-level noise}
     $\tilde{x}'_{t+1} \leftarrow \tilde{x}_t + n_t$  {In-place noise injection}
     $l_t \leftarrow \mathcal{L}(p(y_t|q_t, \tilde{x}_t), y_t)$ 
     $l'_t \leftarrow \mathcal{L}(p(y_t|q_t, \tilde{x}'_{t+1}), y_t)$ 
    if  $l'_t < l_t$  then
         $\tilde{x}_{t+1} \leftarrow \tilde{x}'_{t+1}$  {Accept update if loss is lower}
    else
         $\tilde{x}_{t+1} \leftarrow \tilde{x}_t$  {Discard update otherwise}
    end if
     $t \leftarrow t + 1$ 
end while
    
```

gent (Lee et al., 2024) (long-context reading with episodic memory and gist-level compression), and A-MEM (Xu et al., 2025) (long-term memory via structured notes and dynamic indexing). Each agent is instantiated with three proprietary backbone LLMs: GPT-4o (GPT-4o) (OpenAI, 2024), Gemini 2 Flash Preview (Gemini 2) (Google, 2025), and DeepSeek Chat (DeepSeek) (DeepSeek, n.d.).

Dataset and Metrics. We evaluate each agent on two established benchmarks and report their standard metrics: LessonL uses HumanEval (Chen, 2021) and MBPP (Austin et al., 2021) (pass@1); ReadAgent uses NarrativeQA (Kočíský et al., 2018) (token-level F1) and QUALITY (Pang et al., 2022) (accuracy); A-MEM uses LoCoMo (Maharana et al., 2024) and DialSim (Kim et al., 2024) (token-level F1, open-ended QA).

Baselines. We compare against two controlled baselines (PragLocker ablations), since existing prompt protection/obfuscation methods rely on assumptions mismatched to our black-box untrusted-deployment setting. Specifically,

Table 2. Measuring prompt **non-portability** across different underlying LLMs. For each *Target LLM*, we develop a prompt and then simulate prompt theft by evaluating its performance when copied and reused on other LLMs. We report performance for each agent–task pair under four settings: *Without protection*, *PragLocker (ours)*, and two variants *PragLocker_{tune}* and *PragLocker_{code}*. We report the *Mean Portability Loss*, which is calculated by summing all metrics and expressing it as a multiple of without protection.

Agent	Tasks	Target LLM	Without protection			PragLocker (Ours)			PragLocker _{tune}			PragLocker _{code}		
			GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek
LessonL	HumanEval	GPT-4o	-	98.78	97.56	-	3.04	1.22	-	92.07	90.85	-	95.73	94.51
		Gemini 2	93.90	-	97.56	0.61	-	1.22	85.36	-	89.63	92.07	-	95.12
		DeepSeek	93.90	98.78	-	0.61	2.44	-	82.93	93.29	-	90.24	95.73	-
	MBPP	GPT-4o	-	97.33	94.56	-	1.03	0.72	-	87.26	85.21	-	96.50	93.94
		Gemini 2	91.89	-	94.56	0.51	-	0.62	81.21	-	84.29	91.37	-	93.43
		DeepSeek	91.89	97.33	-	0.62	0.92	-	80.59	87.78	-	91.47	96.71	-
ReadAgent	NarrativeQA	GPT-4o	-	20.81	24.22	-	9.89	10.16	-	12.83	12.43	-	22.61	22.74
		Gemini 2	23.52	-	24.22	10.11	-	10.13	11.46	-	12.32	21.87	-	23.50
		DeepSeek	23.52	20.81	-	9.14	8.07	-	11.10	11.95	-	21.17	23.72	-
	QUALITY	GPT-4o	-	80.90	75.90	-	50.68	53.41	-	65.95	65.86	-	80.23	84.96
		Gemini 2	86.19	-	88.06	52.20	-	54.44	62.63	-	64.54	85.31	-	87.11
		DeepSeek	86.19	75.90	-	51.12	48.56	-	63.03	66.12	-	84.15	80.50	-
A-Mem	LoCoMo	GPT-4o	-	22.85	25.62	-	0.08	0.12	-	15.76	16.13	-	21.73	23.95
		Gemini 2	24.61	-	25.62	0.06	-	0.14	11.98	-	15.22	24.03	-	25.48
		DeepSeek	24.61	22.85	-	0.06	0.10	-	12.87	16.61	-	24.70	22.46	-
	DialSim	GPT-4o	-	2.17	3.62	-	0.04	0.10	-	1.28	2.08	-	2.08	3.41
		Gemini 2	2.86	-	3.62	0.07	-	0.11	1.45	-	2.59	2.46	-	3.53
		DeepSeek	2.86	2.17	-	0.06	0.04	-	1.57	1.36	-	2.79	2.15	-
Cross-LLM Portability Ratio (↓)			1.00×			0.20×			0.82×			0.99×		

Table 3. **Performance preservation** of PragLocker-protected prompts. We compare the original prompt with its PragLocker-protected counterpart on the target LLM.

Agent	Tasks	Without protection			After protection		
		GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek
LessonL	HumanEval	93.90	98.78	97.56	94.51	99.39	98.17
	MBPP	91.89	97.33	94.56	91.99	97.54	94.46
ReadAgent	NarrativeQA	23.52	20.81	24.22	23.61	20.93	25.81
	QUALITY	86.19	75.90	88.06	86.03	76.13	87.97
A-Mem	LoCoMo	24.61	22.85	25.62	25.11	23.01	26.80
	DialSim	2.86	2.17	3.62	2.84	2.31	3.70
Performance Preservation ($\uparrow$)		1.00 $\times$			1.01 $\times$		

PragLocker_{tune} performs only the optimization stage, skipping the initialization transformation; *PragLocker_{code}* applies only the transformation that rewrites the prompt into a code-like representation, without subsequent optimization.

4.2. Main Results

Non-Portability. We first evaluate whether PragLocker resists *cross-model misuse* after prompt theft. We simulate an attacker who obtains the deployed (obfuscated) prompt and reuses it on other LLMs. As shown in Table 2, prompts are highly portable *without protection*: copying a prompt to a different underlying LLM preserves strong performance (e.g., a HumanEval prompt achieves 98.78 on Gemini 2 and 97.56 on DeepSeek). In contrast, *PragLocker makes stolen prompts largely unusable*, driving performance close to zero across agents and tasks (e.g., 3.04/1.22), and achieves the lowest relative mean portability (0.2 $\times$), compared to 1 $\times$ without protection. Moreover, full PragLocker outperforms the simplified baselines: PragLocker_{code} remains nearly as portable as no protection (0.99 $\times$), whereas PragLocker_{tune} only partially reduces portability (0.82 $\times$).

Performance Preservation. We further assess whether PragLocker preserves the agent’s utility by replacing the origi-

nal system prompt with the protected prompt and re-running the same tasks. As Table 3 shows negligible degradation: protected performance closely matches the original across agents/tasks (relative mean 1.01 $\times$ vs. 1 $\times$). *The slight gain likely stems from the feedback-guided tuning stage, which can refine redundant or suboptimal phrasing in the original prompt while maintaining its intended functionality.*

4.3. Case Studies

We present a concise case study illustrating how PragLocker converts a human-readable system prompt into a protected prompt for a keyword-extraction subtask from ReadAgent. We provide an executable code implementation of this case study in the Supplementary Material. Specifically, taking DeepSeek as the target model, the protected prompt preserves utility: it follows the intended instruction and returns relevant keywords in the required JSON structure, matching the behavior of the original prompt.

However, directly transferring the same protected prompt to GPT-4o fails: the model interprets the obfuscated markup as malformed or noisy input. Moreover, the protected prompt is unreadable and structurally perturbed, leaving at most sparse, non-actionable fragments, which makes reconstructing the original prompt from the deployed artifact highly impractical. This challenge compounds for more complex tasks, where obfuscation becomes even more effective.

4.4. Further Analysis

Ablation Study. We conduct a stage-wise ablation to isolate the contributions of *code transformation* and *noise-injected optimization* under the same evaluation protocol. As shown in Table 2, the two stages play distinct roles

Original Prompt	PragLocker Protected Prompt (DeepSeek as target model)
Given the following question, generate several keywords, using ' ' as the separator. // line 1 Question: {How does photosynthesis work?} // line 2 Format your response as a JSON object with a "keywords" field containing the selected text. // line 3 Example response format: // line 4 [{"keywords": "keyword1, keyword2, keyword3"}] // line 5	<pre> <γ-7δ\$4erator="lexosy6thmzseparatorI"cosmy*X>+<input t3pid"quer {" val="{How does photosynthesis work?}"/>_<r6cesG0aode=" keyword-exzract"vcout="\$ynQeiw" scoring="tf4gf"/> // line 1 <Jutput Vo%0qt="\$son"'> // line 2 <:/eld name="k*ywor\$s< type="CBncat<nate9" separatorM"/, VEndVnq="d,rect"/> // line 3 </%utput> // line 4 <exa.p;e-template> // line 5 <json>{"keywo6Rs": "α1, α2, α3"0</json> // line 6 S/exaKple-template> // line 7 B/γ,7δ>x x x -H< x xjxr x // line 8 </pre>
DeepSeek Output	DeepSeek Output
<pre> {"keywords": "photosynthesis, process, plants, sunlight, carbon dioxide, water, glucose, oxygen, chlorophyll, chloroplasts"} </pre>	<pre> {"keywords": "photosynthesis, process, sunlight, chlorophyll, carbon dioxide, water, oxygen, glucose, plants, energy"} </pre>
GPT-4o Output	GPT-4o Output
<pre> {"keywords": "photosynthesis, process, light energy, chlorophyll, carbon dioxide, water, glucose, oxygen, plants"} </pre>	<pre> {It looks like your input includes a mix of structured markup and placeholder-like syntax....} </pre>

Figure 2. Case study: the original prompt and the protected prompt on DeepSeek (target model) vs. GPT-4o; task: keyword extraction.

Table 4. Comparison of inter-family and intra-family portability. Left: inter-family transfer. Right: intra-family transfer within the Qwen2.5 Instruct family (Qwen2.5-7B-Instruct (Qwen-7B) / Qwen2.5-14B-Instruct (-14B) / Qwen2.5-14B-Instruct-bnb-4bit (-14B-4bit)).

Tasks	Target LLM	Original performance	Inter-family portability			Target LLM	Original performance	Intra-family portability		
			GPT-4o	Gemini 2	DeepSeek			Qwen-7B	-14B	-14B-4bit
NarrativeQA	GPT-4o	23.52	-	9.89	10.16	Qwen-7B	16.21	-	8.12	8.97
	Gemini 2	20.81	10.11	-	10.13	-14B	19.36	6.55	-	7.12
	DeepSeek	24.22	9.14	8.07	-	-14B-4bit	18.98	5.96	7.54	-
QUALITY	GPT-4o	86.19	-	50.68	53.41	Qwen-7B	67.60	-	44.57	48.92
	Gemini 2	75.90	52.20	-	54.44	-14B	74.84	43.45	-	46.75
	DeepSeek	88.06	51.12	48.56	-	-14B-4bit	72.07	41.14	43.23	-

and are most effective when combined. *PragLocker_{code}*, which only rewrites the prompt into a code-like representation, offers little protection against cross-model reuse: its mean portability degradation remains $0.99\times$ as code-form prompts are still broadly executable across other LLMs; however, *we interpret this stage as primarily providing a structured initialization that further expands the optimization search space for the subsequent tuning stage*. Specifically, *PragLocker_{tune}* (optimization-only) reduces portability to $0.82\times$ but remains far weaker than full PragLocker ($0.2\times$). Overall, strong resistance requires *both* stages: code transformation supplies an obfuscating scaffold and optimization room, while optimization ultimately converts it into a strongly non-portable prompt representation.

Intra-Family Portability. As PragLocker is optimized from target-LLM I/O feedback, a natural concern is whether behaviorally similar sibling models permit intra-family reuse. We therefore evaluate pairwise portability within the Qwen2.5 Instruct family (Qwen-7B / -14B / -14B-4bit). Table 4 indicates strong non-portability under both scale shifts and quantization shifts, and exhibits no qualitative difference from inter-family portability on the left side of the table: on NarrativeQA, although the original performance is 16.21 (Qwen-7B), 19.36 (-14B), and 18.98 (-14B-4bit), transferring protected prompts across siblings drops

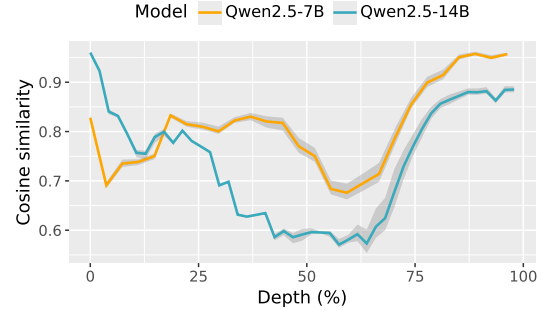


Figure 3. Layerwise hidden state cosine similarity between obfuscated prompt and original prompt; non-portability direction: Qwen2.5-7B → Qwen2.5-14B; 95% confidence interval is shown.

to only 5.96–8.97 (e.g., 7B→14B: 8.12; 14B→7B: 6.55; 14B→14B-4bit: 7.12). We attribute this robustness to our noise-injected optimization objective, which tightly couples the protected prompt to the target model’s idiosyncratic feedback dynamics, so that even modest distributional shifts (scaling or quantization) substantially degrade reuse.

Hidden-State Alignment. To probe the mechanistic origin of non-portability, we compare how the prompt pair (original, obfuscated) is represented inside the target versus a non-target model. We use a protected prompt optimized on Qwen2.5-7B (target) and run the original and obfuscated

Table 5. Adaptive-attack evaluation results. We report task performance under three adaptive attackers: (i) *Deobfuscation*, which optimizes the obfuscated prompt with an inverted objective; (ii) *LLM-assisted recovery*, which queries the target LLM to reconstruct the original prompt; and (iii) *Naive self-prompting*, where the target LLM synthesizes a fresh prompt for itself using only task input–output pairs.

Agent	Tasks	Target LLM	PragLocker (Ours)			LLM-assisted recovery			Deobfuscation			Naive prompt		
			GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek	GPT-4o	Gemini 2	DeepSeek
LessonL	HumanEval	GPT-4o	-	3.04	1.22	-	86.59	87.20	-	3.04	1.83	-	94.51	95.12
		Gemini 2	0.61	-	1.22	82.32	-	85.37	0.61	-	1.22	90.85	-	93.90
		DeepSeek	0.61	2.44	-	84.15	87.02	-	1.22	2.44	-	91.68	95.73	-
	MBPP	GPT-4o	-	1.03	0.72	-	83.16	84.29	-	1.13	1.33	-	90.24	90.45
Gemini 2		0.51	-	0.62	79.57	-	82.34	0.72	-	0.62	85.22	-	90.14	
DeepSeek		0.62	0.92	-	82.03	83.68	-	1.32	0.92	-	85.73	90.97	-	
ReadAgent	NarrativeQA	GPT-4o	-	9.89	10.16	-	5.46	6.87	-	10.21	10.34	-	17.38	21.32
		Gemini 2	10.11	-	10.13	4.64	-	5.29	10.26	-	10.36	19.28	-	21.05
		DeepSeek	9.14	8.07	-	6.71	6.33	-	9.28	8.20	-	21.03	18.85	-
	QUALITY	GPT-4o	-	50.68	53.41	-	21.82	39.06	-	51.13	54.20	-	72.11	83.97
		Gemini 2	52.20	-	54.44	31.82	-	34.57	53.54	-	54.88	82.34	-	82.87
		DeepSeek	51.12	48.56	-	38.56	26.79	-	51.36	51.22	-	84.29	72.61	-
A-Mem	LoCoMo	GPT-4o	-	0.08	0.12	-	14.08	16.23	-	0.39	0.47	-	19.21	22.86
		Gemini 2	0.06	-	0.14	12.89	-	14.94	0.25	-	0.55	18.23	-	20.47
		DeepSeek	0.06	0.10	-	16.43	14.57	-	0.30	0.37	-	22.69	20.33	-
	DialSim	GPT-4o	-	0.04	0.10	-	1.22	1.75	-	0.06	0.15	-	2.07	2.56
Gemini 2		0.07	-	0.11	1.41	-	1.56	0.12	-	0.18	2.13	-	2.29	
		DeepSeek	0.06	0.04	-	1.33	1.31	-	0.10	0.05	-	2.45	2.04	-
Relative Attack Gain (↓)			1×			3.49×			1.03×			4.78×		

prompts separately through the model. At each transformer layer ℓ , we extract their hidden states and compute the cosine similarity between the resulting representations as a layer-wise alignment score. We then feed the identical prompt pair into Qwen2.5-14B (non-target) to repeat the measurement.

Figure 3 reveals a clear *trajectory divergence* across depth. The two models behave similarly in early layers, but their alignment patterns separate as depth increases: after a mid-layer dip, Qwen2.5-7B exhibits a stronger recovery and reaches substantially higher similarity in the later layers, whereas Qwen2.5-14B remains less aligned. This suggests that PragLocker’s functionality preservation is realized through a target-specific internal pathway—its obfuscated prompt is mapped to representations that converge back toward those induced by the original prompt *only* in the target model—providing mechanistic evidence that non-portability is formed predominantly in deeper representations rather than in shallow token-level processing.

Token-Only Feedback. Our default implementation uses token log-probabilities to compute the cross-entropy task loss in Eq. (1). To test whether this feedback is necessary, we evaluate a stricter token-only setting, where the optimizer only observes decoded outputs and uses the task metric as the black-box objective. Since this signal is sparser, we increase the optimization budget to 100 epochs. As shown in Table 6, PragLocker remains effective under token-only feedback with 100 optimization epochs. The optimized prompts preserve target utility, achieving 85.85%, 71.31%, and 82.05% accuracy on GPT-4o, Gemini-2, and DeepSeek, respectively, while still showing clear cross-model degradation. For example, the GPT-4o optimized prompt drops from 85.85% on GPT-4o to 58.16% on Gemini-2 and 63.50% on DeepSeek. These results show that log-probabilities are helpful but not required; PragLocker can operate with token-only black-box feedback at the cost of a larger optimization

Table 6. Token-only feedback evaluation of ReadAgent on QuALITY. The prompt is optimized using only decoded token outputs, without access to token log-probabilities.

Agent	Tasks	Target LLM	GPT-4o	Gemini 2	DeepSeek
ReadAgent	QuALITY	GPT-4o	85.85	58.16	63.50
		Gemini 2	59.67	71.31	60.82
		DeepSeek	58.56	55.24	82.05

budget.

4.5. Adaptive Attacks

LLM-Assisted Prompt Recovery. In this section, we test whether the obfuscated prompt can be recovered at the *text* level. Specifically, we simulate a sophisticated attacker who queries the *target LLM itself* to (i) interpret the obfuscated prompt and reconstruct a natural-language instruction prompt, which is then (ii) substituted back into the agent system (more details are provided in Section D.1).

Strikingly, as shown in Table 5, *even the target LLM itself fails to reliably recover the underlying instructions*, the recovered prompts remain unusable and do not restore transferable functionality. Specifically, LLM-assisted recovery yields only marginal gains over directly reusing the PragLocker-protected prompt (from $1\times$ to $3.49\times$ in relative mean performance). This suggests that, after obfuscation, the protected prompt behaves less like a text-level semantic instruction and more like a *model-conditioned trigger*: its utility is encoded in the target model’s idiosyncratic response dynamics, leaving little recoverable text-level semantics even to the target LLM itself.

Deobfuscation Attack. We also consider a sophisticated attacker who knows our obfuscation pipeline and is given the same task instructions, input–output examples, and evaluation metrics as the defender. Under this setting, the attacker starts from the deployed obfuscated prompt $\hat{x}$ and attempts

to “deobfuscate” it by iteratively re-optimizing the prompt toward a more portable, human-readable instruction. Concretely, the attacker drops $\mathcal{L}_{\text{dist}}$, keeps $\mathcal{L}_{\text{task}}$, flips the sign of $\mathcal{L}_{\text{non-lang}}$, and then iteratively tunes the prompt accordingly to push it back toward the natural-language distribution. For comparison, we also include a *naive prompt* baseline, where the attacker provides the task query and ground-truth outputs and asks the target LLM to synthesize a fresh prompt for solving the task (details of the *deobfuscation* and *naive prompt* attacks are provided in Section D.2).

As shown in Table 5, this attack is largely ineffective: Relative Mean Performance improves by only $1.03\times$, and per-task metrics remain near unusable, far from approaching the unprotected prompt’s portability. Notably, it is even weaker than the naive prompt baseline ($4.78\times$), suggesting that *attackers are better off directly eliciting a new prompt themselves from the LLM than attempting to invert our obfuscation*.

5. Limitation and Discussion

Tokenizer Compatibility. PragLocker injects character-level noise, which requires the target LLM’s tokenizer to have broad character coverage so that perturbed text still maps to valid subword/byte tokens (instead of unknown/rejected tokens). This assumption generally holds for mainstream production API LLMs (e.g., GPT-series (OpenAI, 2022), Google Gemini (Team et al., 2023)).

Prompt-Length Overhead. PragLocker can lengthen the system prompt and increase the prefill cost. In practice, the obfuscated prompt is a fixed prefix, so its KV states can be cached and reused across requests rather than recomputed each time. Thus, the extra cost is largely amortized per cached context, limiting its impact on end-to-end latency.

Comparison with White-Box Defenses. PragLocker is designed for black-box prompt protection in proprietary LLM deployments, where the defender accesses the target model only through APIs. As a result, this work does not extensively compare against prompt-protection methods developed under stronger assumptions, such as white-box gradient access or model adaptation. These methods operate in a partially different regime and are therefore not directly comparable to our setting.

Protection scope. PragLocker protects a specific and practically important component of agent IP: deployed prompts, which often encode task instructions, control logic, and format constraints in prompt-based agents. Its goal is to reduce the direct cross-model reusability of leaked prompts, rather than eliminate all routes to reproducing agent behavior. Broader behavior-reproduction attacks, such as imitation, prompt induction, or distillation from examples, follow a different attack path and are therefore largely orthogonal to PragLocker.

6. Conclusions

We study system-prompt theft in untrusted LLM-agent deployments, where a leaked prompt can be readily reused across models. Specifically, we identify prompts as the primary embodiment of agent IP and distill four requirements for protection. We propose PragLocker, a black-box protection method that transforms a prompt into a model-conditioned, non-portable form via structured initialization and noise-injected optimization. Experiments show that PragLocker strongly reduces portability to other LLMs, remaining robust to LLM-assisted recovery attempts.

Acknowledgements

This work was supported by the Key R&D Program of Ningbo under Grant No.2024Z115.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Andriushchenko, M., Croce, F., and Flammarion, N. Jail-breaking leading safety-aligned LLMs with simple adaptive attacks. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=hXA8wqRdyV>.
- Anthropic. Giving claude a role with a system prompt (claude docs), 2025. URL <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/system-prompts>.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Chang, K., Xu, S., Wang, C., Luo, Y., Liu, X., Xiao, T., and Zhu, J. Efficient prompting methods for large language models: A survey. *arXiv preprint arXiv:2404.01077*, 2024.
- Chen, M. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cloud, H. The nist definition of cloud computing. *National institute of science and technology, special publication*, 800(2011):145, 2011.
- Costan, V. and Devadas, S. Intel sgx explained. *Cryptology ePrint Archive*, 2016.

- Cursor. Features · cursor, 2025. URL <https://cursor.com/features>.
- DeepSeek. Deepseek chat, n.d. URL <https://chat.deepseek.com/>. Accessed: 2026-01-28.
- Deng, M., Wang, J., Hsieh, C.-P., Wang, Y., Guo, H., Shu, T., Song, M., Xing, E., and Hu, Z. Rlprompt: Optimizing discrete text prompts with reinforcement learning. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pp. 3369–3391, 2022.
- Friedman, N. Introducing GitHub copilot: your AI pair programmer, June 2021. URL <https://github.com/features/copilot>. Updated Feb 23, 2022.
- Gim, I., Li, C., and Zhong, L. Confidential prompting: Protecting user prompts from cloud llm providers. *arXiv preprint arXiv:2409.19134*, 2024.
- Google. Gemini models | gemini api | google ai for developers, 2025. URL <https://ai.google.dev/gemini-api/docs/models>. Last updated 2025-12-18 (UTC). Accessed 2025-12-25.
- Hui, B., Yuan, H., Gong, N., Burlina, P., and Cao, Y. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pp. 3600–3614, 2024.
- Karvandi, M. S., Meghdadizanjani, S., Arasteh, S., Monfared, S. K., Fallah, M. K., Gorgin, S., Lee, J.-A., and van der Kouwe, E. The reversing machine: reconstructing memory assumptions. *arXiv preprint arXiv:2405.00298*, 2024.
- Kim, J., Chay, W., Hwang, H., Kyung, D., Chung, H., Cho, E., Jo, Y., and Choi, E. Dialsim: A real-time simulator for evaluating long-term multi-party dialogue understanding of conversation systems. *arXiv preprint arXiv:2406.13144*, 2024.
- Kočiskỳ, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328, 2018.
- Lcvenshtcin, V. Binary coors capable or ‘correcting deletions, insertions, and reversals. In *Soviet physics-doklady*, volume 10, 1966.
- Lee, K.-H., Chen, X., Furuta, H., Canny, J., and Fischer, I. A human-inspired reading agent with gist memory of very long contexts. *arXiv preprint arXiv:2402.09727*, 2024.
- Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.
- Li, Z., Liu, Y., Su, Y., and Collier, N. Prompt compression for large language models: A survey. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7182–7195, 2025.
- Lin, S., Hua, W., Wang, Z., Jin, M., Fan, L., and Zhang, Y. Emojiprompt: Generative prompt obfuscation for privacy-preserving communication with cloud-based llms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 12342–12361, 2025.
- Liu, Y., Deng, R., Kaler, T., Chen, X., Leiserson, C. E., Ma, Y., and Chen, J. Lessons learned: A multi-agent framework for code llms to learn and improve. *arXiv preprint arXiv:2505.23946*, 2025.
- Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.
- Manus AI. Manus: Hands on ai. <https://manus.im/>, 2026. Accessed: 2026-01-08.
- McGovern, H. E., Stureborg, R., Suhara, Y., et al. Your large language models are leaving fingerprints. In *Proceedings of the 1st Workshop on GenAI Content Detection (GenAIDetect)*, pp. 85–95, 2025.
- Nertney, R. Confidential compute on nvidia hopper h100. Technical Report WP-11459-001, NVIDIA, July 2023. URL <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/HCC-Whitepaper-v1.0.pdf>. Version 1.0. Accessed 2025-12-25.
- Nikolaou, G., Mencattini, T., Crisostomi, D., Santilli, A., Panagakis, Y., and Rodolà, E. Language models are injective and hence invertible. *arXiv preprint arXiv:2510.15511*, 2025.
- OpenAI. Introducing chatgpt, November 2022. URL <https://openai.com/index/chatgpt/>.
- OpenAI. Hello gpt-4o, May 2024. URL <https://openai.com/index/hello-gpt-4o/>. Accessed: 2026-01-28.
- OpenAI. Api reference: Introduction, 2025a. URL <https://platform.openai.com/docs/api-reference/introduction>. Accessed 2025-12-25.
- OpenAI. Text generation (openai api guide), 2025b. URL <https://platform.openai.com/docs/guides/text>.

- Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., et al. Quality: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5336–5358, 2022.
- Pape, D., Mavali, S., Eisenhofer, T., and Schönherr, L. Prompt obfuscation for large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 2323–2342, 2025.
- Rana, S., Khoda Parast, F., Kelly, B., Wang, Y., and Kent, K. B. A comprehensive survey of cryptography key management systems. *Journal of Information Security and Applications*, 74:103607, 2023. doi: 10.1016/j.jisa.2023.103607.
- Rastrigin, L. The convergence of the random search method in the extremal control of a many parameter system. *Automaton & Remote Control*, 24:1337–1342, 1963.
- Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., and Chadha, A. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024. URL <https://arxiv.org/abs/2402.07927>.
- Salimans, T., Ho, J., Chen, X., Sidor, S., and Sutskever, I. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- Schwefel, H.-P. Evolutionsstrategien für die numerische optimierung. In *Numerische Optimierung von Computer-Modellen Mittels der Evolutionsstrategie: Mit Einer Vergleichenden Einführung in Die Hill-Climbing-und Zufallsstrategie*, pp. 123–176. Springer, 1977.
- Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E., and Singh, S. Autoprompt: Eliciting knowledge from language models with automatically generated prompts. *arXiv preprint arXiv:2010.15980*, 2020.
- Singh, C., Morris, J. X., Aneja, J., Rush, A. M., and Gao, J. Explaining data patterns in natural language with language models. In *Proceedings of the 6th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 31–55, 2023.
- Spector, S. Zapier agents: Work hand in hand with AI agents, November 2025. URL <https://zapier.com/blog/zapier-agents-guide/>. Most recently updated Nov 2025.
- Su, J. and Zhang, W. Runtime attestation for secure llm serving in cloud-native trusted execution environments. In *Machine Learning for Computer Architecture and Systems 2025*.
- Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- The Kubernetes Authors. Encrypting confidential data at rest, May 2025. URL <https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>. Last modified May 09, 2025. Accessed 2025-12-25.
- Wang, J., Yang, T., Xie, R., and Dhingra, B. Raccoon: Prompt extraction benchmark of llm-integrated applications. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 13349–13365, 2024a. URL <https://aclanthology.org/2024.findings-acl.791/>.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J.-R. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024b. doi: 10.1007/s11704-024-40231-1.
- Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- Yang, Y., Li, C., Li, Q., Ma, O., Wang, H., Wang, Z., Gao, Y., Chen, W., and Ji, S. {PRSA}: Prompt stealing attacks against {Real-World} prompt services. In *34th USENIX security symposium (USENIX Security 25)*, pp. 2283–2302, 2025a.
- Yang, Y., Li, Y., Yao, H., Huang, E., Shao, S., Wang, Y., Wang, Z., Tao, D., and Qin, Z. Promptcos: Towards content-only system prompt copyright auditing for llms. *arXiv preprint arXiv:2509.03117*, 2025b.
- Yao, H., Lou, J., Qin, Z., and Ren, K. Promptcare: Prompt copyright protection by watermark injection and verification. In *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 845–861. IEEE, 2024.
- Yuan, M., Zhang, L., Zeng, L., Jiang, S., Yang, B., Duan, D., and Xing, G. Scx: Stateless kv-cache encoding for cloud-scale confidential transformer serving. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pp. 39–54, 2025.
- Zapier. Build AI teammates with zapier agents, 2025. URL <https://zapier.com/agents>.
- Zhang, W., Tang, K., Wu, H., Wang, M., Shen, Y., Hou, G., Tan, Z., Li, P., Zhuang, Y., and Lu, W. Agent-pro: Learning to evolve via policy-level reflection and optimization. *arXiv preprint arXiv:2402.17574*, 2024.

Appendix

▶ A Discussions on Theoretical Motivation for PragLocker	13
▶ B Details on Main Experiment	15
▶ C Details on Case Study	16
▶ D Details on Adaptive Attacks	16
▷ D.1 LLM-Assisted Prompt Recovery.	16
▷ D.2 Deobfuscation Attack.	17
▶ E Discussions on PragLocker Methodology	17

Table 7. Glossary.

Symbol	Meaning
$d \in \mathbb{R}$	Model dimension.
$\mathcal{V}$	Model vocabulary.
$\mathcal{P} \subset \{\mathcal{V}, \mathcal{V}^2, \dots\}$	Prompt space.
$\mathbf{x} \in \mathcal{P}$	Original agent prompt.
$\tilde{\mathbf{x}} \in \mathcal{P}$	Obfuscated prompt for target prompt $\mathbf{x}$.
$\text{Embed}(\cdot)$	Embedding layer.
$\mathbf{h} \in \mathbb{R}^d$	Model embeddings; $\mathbf{h} = \text{Embed}(\mathbf{x})$.
δ	Perturbation to embeddings.
$\mathbf{q}$	An instance of task query, often used together with the agent prompt.
$\mathcal{Q}$	Set of task queries.
$f(\cdot) \in \mathbb{R}^{ \mathcal{V} }$	Model output logits.
$y \in \mathcal{V}$	An output token; $y \sim f(\cdot)$.
θ	Model parameters.
$m(\cdot)$	Correct-class margin function.
$S_{\mathbf{x}}$	Stability region of prompt $\mathbf{x}$.
$B_{\epsilon}(\mathbf{h})$	ϵ -ball of embeddings $\mathbf{h}$; $B_{\epsilon}(\mathbf{h}) = \{\mathbf{h}' \ \mathbf{h}' - \mathbf{h}\ _2 < \epsilon\}$.
$d(\cdot, \cdot)$	Distance between two prompts; edit distance by default.
$\text{Init}(\cdot)$	Prompt initialization for obfuscation optimization.
$\mathcal{L}(\cdot)$	Objective function for obfuscation optimization.

A. Discussions on Theoretical Motivation for PragLocker

In this section, we provide details that extend our discussions in Section 3.2. We will show the existence of obfuscated prompts that satisfy our criteria of obfuscation, utility and non-portability.

Roadmap. We start with defining the notion of “functional equivalence” in the setting of single-token generation (*utility of embeddings*). We show that there always exists embeddings that is different from yet functionally equivalent to embeddings of the original prompt (*local obfuscation and utility*). We then show that an obfuscated prompt satisfying the utility constraint often fails to transfer to another model with parameters different from the target model (*non-portability*). Finally, we extend our theoretical results to the scenario of open-ended, autoregressive generation.

Let output logits of the target model be $f(\cdot; \theta)$. Let original agent prompt embeddings be $\mathbf{h} = \text{Embed}(\mathbf{x}) \in \mathbb{R}^{n \times d}$, i.e. embeddings with length $n := |\mathbf{x}|$ and dimension d . The obfuscated embeddings ($\tilde{\mathbf{h}}$) can be obtained by adding noise δ to $\mathbf{h}$, such that $\tilde{\mathbf{h}} = \mathbf{h} + \delta$. We assume $\tilde{\mathbf{x}}$ has the same length as $\mathbf{x}$, which can always be achieved via padding either $\tilde{\mathbf{x}}$ or $\mathbf{x}$ at initialization.

For the moment, we focus on embeddings in continuous space, not prompts in discrete space.

By saying that embeddings $\tilde{\mathbf{h}}$ is functionally equivalent to $\mathbf{h}$, we refer to sampled model responses: for $\forall \mathbf{q}_i, \tilde{y}_i \sim f(\cdot | \tilde{\mathbf{h}}, \mathbf{q}_i), y_i \sim f(\cdot | \mathbf{h}, \mathbf{q}_i)$ where $\tilde{y}_i, y_i \in \mathcal{V}$, we have $\tilde{y}_i = y_i$. We adopt a relaxed definition of functional equivalence based on discrete output behavior and restrict sampling strategy to greedy decoding:

Definition A.1 (Functional equivalence of embeddings). We say the embeddings $\tilde{\mathbf{h}}$ and $\mathbf{h}$ of two prompts ($\mathbf{x}, \tilde{\mathbf{x}}$) are functionally equivalent with respect to a query $\mathbf{q}_i$ if the same outputs are obtained:

$$\tilde{y}_i = \arg \max_y f(y | \tilde{\mathbf{h}}, \mathbf{q}_i), y_i = \arg \max_y f(y | \mathbf{h}, \mathbf{q}_i). \quad (2)$$

This definition is less restrictive and more practical than representation-level equivalence, since in practice we do not care about hidden states, only final discrete outputs.

Following our definition of functional equivalence, we now establish the conditions under which such equivalent prompts exist and satisfy our objectives.

Local obfuscation and utility. To prove that we can find an obfuscated prompt that preserves utility, we first establish the geometric properties of the model’s decision boundary in the embedding space.

Lemma A.2 (Equivalent condition for functional equivalence). *Let $y_i = \arg \max_y f(y | \mathbf{h}, \mathbf{q}_i)$ be the target output for the original prompt. For an obfuscated embedding $\tilde{\mathbf{h}}$ to be functionally equivalent to $\mathbf{h}$, it is sufficient that the correct-class*

margin is strictly positive:

$$m(\tilde{\mathbf{h}}, \mathbf{q}_i, y_i) := f(\tilde{\mathbf{h}}, \mathbf{q}_i)_{y_i} - \max_{k \neq y_i} f(\tilde{\mathbf{h}}, \mathbf{q}_i)_k > 0$$

Proof. If $m(\tilde{\mathbf{h}}, \mathbf{q}_i, y_i) > 0$, then $f(\tilde{\mathbf{h}}, \mathbf{q}_i)_{y_i} > f(\tilde{\mathbf{h}}, \mathbf{q}_i)_k$ for all $k \neq y_i$. Consequently, the argmax operation under greedy decoding yields $\tilde{y}_i = y_i$, satisfying Definition A.1. $\square$

We next show that this condition holds not just for a single point, but for a region surrounding .

Lemma A.3 (Continuity of the margin function). *The margin function $m(\mathbf{h}, \mathbf{q}_i, y_i)$ is continuous with respect to the input embeddings $\mathbf{h}$.*

Proof. The neural network $f(\cdot; \theta)$ is a composition of continuous functions (linear transformations, layer normalizations, and activations like GeLU/Softmax). The operations $\max(\cdot)$ and subtraction are also continuous. Therefore, their composition $m(\cdot)$ is continuous with respect to $\mathbf{h}$. $\square$

Using continuity, we define the region in embedding space where utility is preserved.

Definition A.4 (Stability region). We define the stability region $S_{\mathbf{x}}$ as the set of all embeddings that maintain positive margins for the target outputs across a query set $\mathcal{Q}$:

$$S_{\mathbf{x}} = \{\mathbf{h}' \mid \forall \mathbf{q}_i \in \mathcal{Q}, m(\mathbf{h}', \mathbf{q}_i, y_i) > 0\}$$

Since $m(\mathbf{h}, \dots) > 0$ for the original prompt (assuming the model is confident) and m is continuous, there exists an $\epsilon > 0$ such that the open ball $B_\epsilon(\mathbf{h}) \subset S_{\mathbf{x}}$. Any embedding vector within this ball preserves utility.

Theorem A.5 (Existence of obfuscated prompts). *Given a task prompt $\mathbf{x}$, there exists a prompt $\tilde{\mathbf{x}} \neq \mathbf{x}$ such that:*

1. $\tilde{\mathbf{x}}$ is functionally equivalent to $\mathbf{x}$ (Utility).
2. The distance in prompt space $d(\tilde{\mathbf{x}}, \mathbf{x}) \geq d_0$ for some constant $d_0 > 0$ (Obfuscation).

Proof. Let $\Delta \mathbf{h}$ be the perturbation in embedding space caused by modifying $\mathbf{x}$ to $\tilde{\mathbf{x}}$. We require $\mathbf{h} + \Delta \mathbf{h} \in B_\epsilon(\mathbf{h})$ to satisfy utility.

The total perturbation can be decomposed into token-wise perturbations. Let $\tilde{\mathbf{x}}$ differ from $\mathbf{x}$ at a set of indices $\mathcal{K}$. The norm of the embedding shift is bounded by the sum of individual token shifts: $\|\Delta \mathbf{h}\| \leq \sum_{j \in \mathcal{K}} \|\delta_j\|$.

Due to the attention mechanism in Transformer architectures, specific tokens often exhibit low sensitivity (gradient $\nabla_{\mathbf{x}_j} \mathcal{L} \approx 0$), particularly in long contexts (attention dilution). We can select a set of k such tokens to replace or perturb such that $\sum_{j=1}^k \|\delta_j\| < \epsilon$.

While the embedding perturbation is bounded within ϵ (preserving utility), the discrete prompt distance (e.g., edit distance) $d(\tilde{\mathbf{x}}, \mathbf{x})$ increases with k . By choosing sufficiently many low-sensitivity tokens, we satisfy $d(\tilde{\mathbf{x}}, \mathbf{x}) \geq d_0$ while ensuring $\tilde{\mathbf{h}} \in S_{\mathbf{x}}$. $\square$

Non-portability. Finally, we address why this utility does not transfer to other models.

Proposition A.6 (Non-portability via manifold mismatch). *An obfuscated prompt $\tilde{\mathbf{x}}$ that satisfies utility on model θ is likely to fail on a distinct model θ' .*

Proof. The stability region $S_{\mathbf{x}}(\theta)$ depends on the specific parameters θ . For a different model θ' , the decision boundaries and resulting stability region $S_{\mathbf{x}}(\theta')$ generally do not align with $S_{\mathbf{x}}(\theta)$ in the high-dimensional embedding space.

The perturbation δ constructed in Theorem 1 is optimized specifically to remain within $B_\epsilon(\mathbf{h}) \subset S_{\mathbf{x}}(\theta)$. Without knowledge of θ' , there is no guarantee that $\mathbf{h} + \delta \in S_{\mathbf{x}}(\theta')$. Empirically, adversarial or obfuscated perturbations are known to be sensitive to the specific curvature of the loss landscape, leading to $P(\tilde{\mathbf{h}} \in S_{\mathbf{x}}(\theta')) \ll P(\tilde{\mathbf{h}} \in S_{\mathbf{x}}(\theta))$, ensuring non-portability. $\square$

Prompt template to encode original prompt into code-symbol format for prompt initialization.

```
You are a large language model with strong pattern abstraction ability.
Your task is NOT to execute a prompt, but to DESIGN a prompt DSL that YOU can reliably interpret and execute.
Constraints:
1. The DSL must preserve the intent and usage effect of the original prompt template.
2. The DSL must be directly usable (no intermediate explanation required at runtime).
3. The DSL should rely on structural patterns.
4. The DSL should be difficult for humans to read directly.
5. Do not reduce too much semantic information.
6. Such as """<transmute operator="textflow" mode="bidirectional" direction="input→en">
  <source lang="auto" detect="true">{{QUERY}}</source>
  <constraint preserve="semantic_density" threshold="0.92"/>
  <output format="natural" register="neutral"/>
</transmute>"""

Prompt:
"""{target_prompt}"""
```

Figure 4. Prompt template for the target LLM to rewrite the original prompt into code-symbol format with XML-like structure.

Extension to autoregressive generation. We previously focus on generation of a single token; we will now generalize our results to the practical scenario of open-ended generation.

Proposition A.7 (Autoregressive extension). *The guarantees of utility and obfuscation extend from single-token generation to open-ended autoregressive generation.*

Proof. We proceed by induction on the generation step t .

Base case ($t = 0$): Theorem 1 guarantees $\tilde{y}_0 = y_0$ given $\tilde{\mathbf{x}}$.

Inductive step: Assume $\tilde{y}_k = y_k$ for all $k < t$. At step t , the input context for the obfuscated prompt is $[\tilde{\mathbf{x}}, y_0, \dots, y_{t-1}]$. Since the generated history is identical, the deviation in embeddings stems solely from the initial $\tilde{\mathbf{x}}$. As established in Theorem 1, provided the initial perturbation is within the stability region, the subsequent token prediction remains invariant. Thus, $\tilde{y}_t = y_t$. $\square$

B. Details on Main Experiment

In this section, we provide implementation details for the experiments of Section 4.2.

Prompt initialization: Code-symbol encoding. In Figure 4, we show the prompt template for the target LLM to encode the original prompt into a code-symbol format that is not written in natural language.

Noise scheduling strategy. In the main body, we do not include the noise scheduling strategy for simplicity. However, in practice, we find that *noise annealing* is helpful for obfuscation optimization. We interpret this strategy as an implementation detail rather than our core design. Noise annealing is motivated by our empirical observation that it is hard to inject large amounts of noise at late stages of obfuscation optimization. We hypothesize that obfuscation optimization should be conducted at finer granularities at later stages, whereas aggressive noise injection at earlier stages is helpful for improving efficiency. Therefore we draw inspiration from *learning rate annealing*, a prevailing practice in neural network optimization with stochastic gradient descent.

Specifically, we start with a preset number of random characters to inject sampled noise into, which we call the *initial noise size* (analogous to initial learning rate). After each epoch, we adopt a linear schedule and decrease the number of noise by a constant, which we term the *noise schedule rate* (analogous to learning rate decay rate). We also designate a *minimum noise size* such that optimization does not stop completely at late stages of obfuscation optimization.

Caveat on obfuscation optimization on API LLMs. In practice, obfuscation optimization is hindered by the lack of acc to full logprobs for API LLMs such as GPT-4o and DeepSeek-chat. For example, DeepSeek API¹ only returns the top logprobs of at most 20 tokens (top_logprobs argument). This slightly complicates the computation of the task loss.

¹<https://api-docs.deepseek.com/api/create-chat-completion>

Table 8. Hyperparameters for PragLocker in main experiment.

Hyperparameter	Value
λ	0.1
γ	0.1
Epochs	50
Candidates per epoch	20
Noise set	ASCII characters (0–127)
Initial noise size	$\frac{1}{4}$ prompt length
Noise schedule rate	8
Minimum noise size	4
Top- k logprobs	10
Default logprob	-100.0

The algorithm for computing cross-entropy task loss is shown in Algorithm 2. We set a default logprob (often -100.0) for label tokens missing from the returned top- k logprobs. We take advantage of the top-logprobs functionality of API LLMs while balancing efficiency and performance. As is shown in Table 8, we let $k = 10$ since a large k can greatly slow down API response rate while a small k decreases the precision of loss computation.

Algorithm 2 Computation of task loss for API LLMs.

Input: Base LLM $p(\cdot)$, obfuscated prompt $\tilde{\mathbf{x}}_t$ at the t -th step, query $\mathbf{q}_t$ at the t -th step, labels $\mathbf{y}_t$ at the t -th step, number of top logprobs k , default logprob l_{def}

Output: Task loss l_t

$\text{logps} \leftarrow p(\cdot | \mathbf{q}_t, \tilde{\mathbf{x}}_t)$

$l_t \leftarrow 0$

$i \leftarrow 1$

while $i \leq |\mathbf{y}_t|$ **do**

if $\mathbf{y}_t[i]$ in top- k logprobs **then**

$l_t \leftarrow l_t - \text{logps}[i][\mathbf{y}_t[i]]$

else

$l_t \leftarrow l_t - l_{\text{def}}$

end if

$i \leftarrow i + 1$

end while

$l_t \leftarrow l_t / |\mathbf{y}_t|$

Hyperparameters. We show the hyperparameters of our main experiment in Table 8.

C. Details on Case Study

Reproducibility. In Section 4.3 of the main body, we present an obfuscated prompt and its corresponding original prompt as well as model responses from the target model and non-target model. In order to facilitate reproduction and to support the authenticity of our results, we have provided **an executable demonstration of the case study in the supplementary material**. We encourage readers to execute the Python script for themselves, which requires access to the DeepSeek API².

D. Details on Adaptive Attacks

In this section, we provide implementation details for the adaptive attack experiments of Section 4.5.

D.1. LLM-Assisted Prompt Recovery

We show the prompt template for LLM-assisted prompt recovery in Figure 5. The LLM is provided with the obfuscated prompt and instructed to guess the original prompt. Although we acknowledge that this implementation is simple, we

²<https://api-docs.deepseek.com/>

Prompt template for LLM-assisted prompt recovery.

What is the original prompt for the following prompt?

Prompt:
 """{obfuscated_prompt}"""

Figure 5. Prompt template for LLM-assisted prompt recovery.

Prompt template for naive prompt baseline (LLM-assisted prompt induction).

You are given a small set of input-output examples produced by an agent.
 Your task is to write a SINGLE system prompt that, when used to initialize an LLM agent,
 will reproduce the same behavior on similar inputs.

[Optional] Task context:
 {task_description}

Examples:
 {io_pairs}
 # Each example should follow:
 # Input: ...
 # Output: ...

Requirements for the system prompt:

- Output ONLY the system prompt text (no explanations, no markdown).
- The prompt should specify the agent's role, goals, constraints, and tool-use policy if needed.
- Be concise but complete; preserve the behavior implied by the examples.
- Do NOT refer to these examples explicitly in the final prompt.

System prompt:

Figure 6. Template used to construct the naive prompt baseline via LLM-assisted prompt induction from observed input-output behavior.

emphasize that this strategy directly supports our claim that even the target LLM for which the obfuscated prompt was optimized fails to fully interpret the obfuscated prompt.

D.2. Deobfuscation Attack

Deobfuscation optimization. We show the hyperparameters for deobfuscation attack in Table 9. All hyperparameters are consistent with those of obfuscation optimization, except for the coefficient of the optimization objective. The objective function for the deobfuscation attack includes only $\mathcal{L}_{\text{task}}$ and $\mathcal{L}_{\text{non-lang}}$ terms, not $\mathcal{L}_{\text{dist}}$, since the original prompt is inaccessible for the attacker. We also invert the coefficient for the $\mathcal{L}_{\text{non-lang}}$ term since the attacker aims to recover the original prompt written in natural language.

Naive Prompt Baseline. We also consider a naive baseline where the attacker re-synthesizes a system prompt from task behavior, rather than deobfuscating the deployed prompt. Specifically, the attacker collects a small set of representative input-output pairs via black-box queries and uses an LLM to draft a prompt that reproduces the observed functionality. The drafted prompt is evaluated directly without any adaptive tuning or optimization. The concrete induction template used to generate the naive prompt is provided in Figure 6.

E. Discussions on PragLocker Methodology

On PragLocker vs. prompt compression techniques. Historically, prompt compression is motivated by maintaining task performance comparable to original prompts with less memory and computational cost. PragLocker might be reminiscent of *hard prompt compression methods*, according to the taxonomy of Li et al. (2025). However, these methods focus on removing redundancy from the original prompts and often require auxiliary compression models. Furthermore, hard prompt compression methods do not take obfuscation into consideration. All these properties clearly distinguish PragLocker from

Table 9. Hyperparameters for deobfuscation attack.

Hyperparameter	Value
γ	-0.1
Epochs	50
Candidates per epoch	20
Noise set	ASCII characters (0–127)
Initial noise size	$\frac{1}{4}$ prompt length
Noise schedule rate	8
Minimum noise size	4
Top- k logprobs	10

prompt compression techniques.

Concerns of invertibility. A recent paper claims that transformer language models are almost surely injective and thus invertible (i.e. to recover the input token sequence given hidden state activations) (Nikolaou et al., 2025), which seems to challenge our prompt protection method. However, we argue that the arguments of this paper do not fundamentally undermine the validity of PragLocker; instead, **it strengthens the security of PragLocker**.

Given a task query $\mathbf{q}$ and a prompt $\mathbf{x}$, there are countless hidden states that yield the same greedy output as $\mathbf{h} = f_{<l}(\mathbf{x})$ where $f_{<l}(\cdot)$ is the first l layers of the LM, since the latent space is continuous and the LM has inherent robustness with respect to infinitesimal perturbations to hidden states. All these functionally equivalent hidden states belong to a set of activations: $B_\epsilon = \{\mathbf{h}' \mid \|\mathbf{h}' - \mathbf{h}\|_2 < \epsilon\}$, where $\epsilon > 0$ is a small constant that ensures greedy model outputs are the same. Suppose it is possible to trace back each hidden state of B_ϵ back to input token sequences, then there are infinite number of prompts (assuming model context size is intractably large and we use paddings to handle various prompt lengths) that lead to the same model outputs, since $|B_\epsilon|$ is infinite. This makes the prompt inversion attack intractable, since it is impossible for the attacker to identify the prompt $\mathbf{x}$ among the almost infinite set of functionally equivalent prompts.